**Chapter 4 previous questions**
**Session: 2020-21**
**Course Code: CIT-321 Course Title: Operating System**

---

**2(c)** Identify the basic components of a thread, and contrast threads and processes. Describe the major benefits and significant challenges of designing multithreaded processes. **(3)**

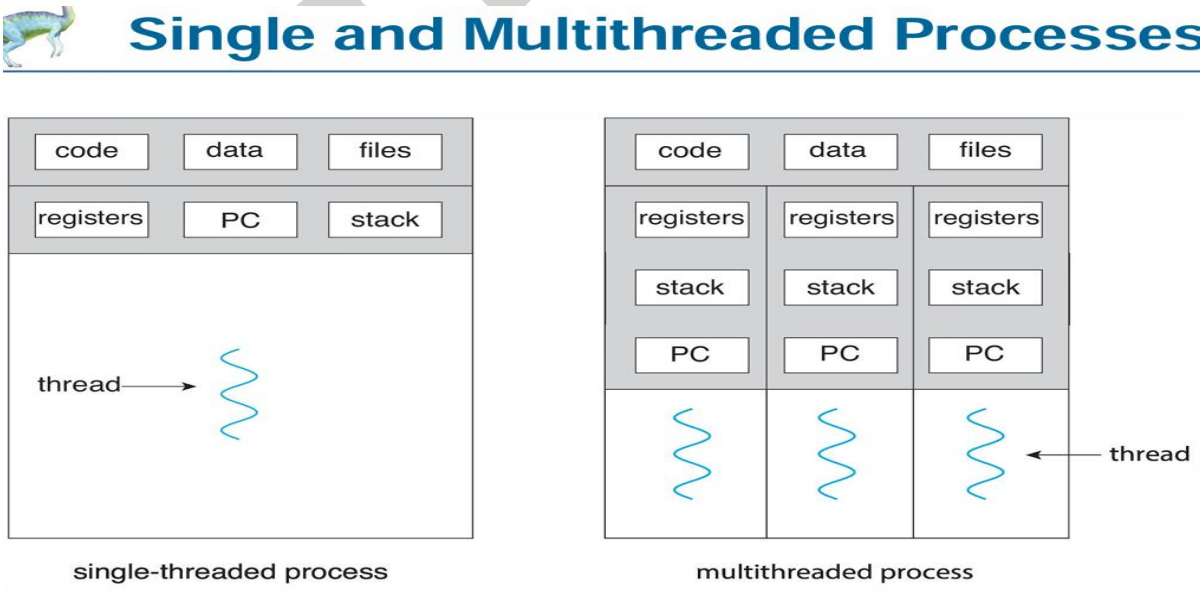
# Basic Components of a Thread

A **thread** is the smallest unit of CPU execution within a process. Multiple threads can exist inside a single process and share the same resources.

Thread হলো একটি process-এর ভিতরের সবচেয়ে ছোট execution unit। একই process-এর মধ্যে একাধিক thread একসাথে কাজ করতে পারে এবং একই resources share করে।

## Components of a Thread

1. **Thread ID (TID)**
   - Unique identification number of the thread.
2. **Program Counter (PC)**
   - Stores the address of the next instruction to execute.
3. **Register Set**
   - Holds temporary data and CPU execution information.
4. **Stack**
   - Stores local variables, function calls, and return addresses.



# Difference Between Threads and Processes

| Feature | Process | Thread |
|---|---|---|
| Definition | Independent program in execution | Small execution unit inside a process |
| Memory | Has separate memory space | Shares memory with other threads |
| Resource Sharing | Resources are separate | Resources are shared |
| Communication | Slower IPC required | Faster communication |
| Creation Time | More time required | Less time required |
| Context Switching | Costly | Faster and cheaper |

**DRMC**

# Benefits of Multithreaded Processes (FEBRI'Q)

## 1. Improved Responsiveness

- One thread can continue running while another waits for I/O operations.

## 2. Resource Sharing

- Threads share memory and resources efficiently.

## 3. Faster Execution

- Multiple tasks can run concurrently.

## 4. Better CPU Utilization

- Multiple processors or CPU cores can execute threads simultaneously.

## 5. Economical

- Creating threads requires less overhead than creating processes.

# Challenges of Multithreaded Processes

## 1. Synchronization Problems

- Multiple threads accessing shared data may cause inconsistencies.

## 2. Race Condition

- Output may become unpredictable when threads modify shared data simultaneously.

## 3. Deadlock

- Threads may wait indefinitely for resources held by each other.

## 4. Difficult Debugging

- Errors in multithreaded programs are harder to detect and fix.

## 5. Security and Data Corruption

- Shared memory may lead to accidental modification of important data.

---

# Conclusion

Threads are lightweight execution units within a process that improve performance and responsiveness. Although multithreading provides efficient resource sharing and faster execution, it also introduces challenges like synchronization issues, race conditions, and deadlocks.

**2(d)** What resources are used when a thread is created? How do they differ from those used when a process is created? Illustrate different approaches to implicit threading, including thread pools, fork-join, and Grand Central Dispatch. **(4)**

# Resources Used When a Thread is Created

When a **thread** is created, it uses only a small amount of resources because it shares most resources with other threads of the same process.

## Resources Used by a Thread

- Thread ID
- Program Counter
- Register Set
- Stack space
- Thread state information

### Shared Resources

Threads of the same process share:

- Code section
- Data section
- Heap
- Open files
- Signals and OS resources

---

# Resources Used When a Process is Created

A **process** requires more resources because it is an independent execution unit.

## Resources Used by a Process

- Separate memory space
- Process Control Block (PCB)
- Code section
- Data section
- Heap and stack
- Open files and I/O resources
- Security and protection information

---

# Difference Between Thread and Process Creation

| Feature | Thread Creation | Process Creation |
|---|---|---|
| Memory Space | Shared | Separate |
| Resource Requirement | Small | Large |
| Creation Time | Faster | Slower |
| Communication | Easy and fast | IPC required |
| Context Switching | Less overhead | More overhead |

---

# Implicit Threading

**Implicit threading** means the operating system or compiler creates and manages threads automatically, reducing the burden on programmers.

Implicit threading হলো এমন একটি পদ্ধতি যেখানে programmer-কে আলাদা করে thread তৈরি ও manage করতে হয় না। Operating system বা compiler নিজে থেকেই threads তৈরি, schedule এবং control করে।

মানে, তুমি task দাও, আর system নিজে ঠিক করে কতগুলো thread লাগবে এবং কীভাবে run হবে।

---

# Approaches to Implicit Threading

## 1. Thread Pools

A **thread pool** is a collection of pre-created worker threads. ( Thread pool হলো আগে থেকেই তৈরি করা কিছু worker thread-এর collection।)

### Working Principle

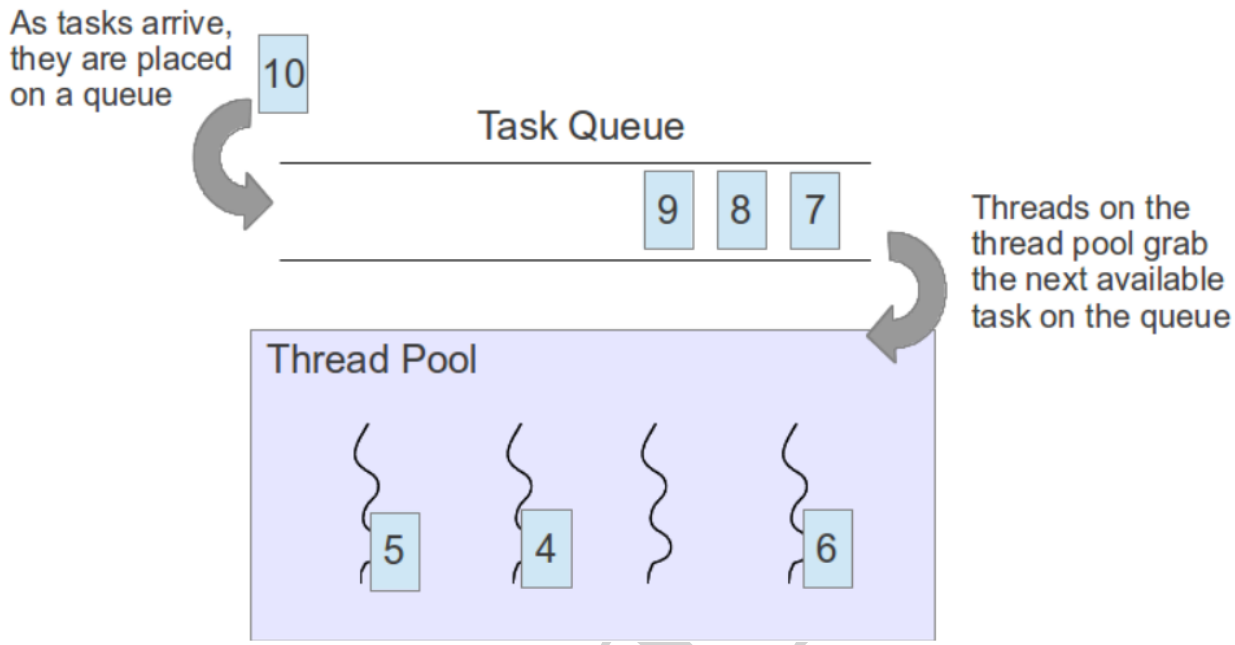
- Tasks are placed in a queue.
- An available thread from the pool executes the task.

### Advantages

- Reduces thread creation overhead

- Improves performance
- Controls the number of threads

### Diagram




সহজ উদাহরণ

ধরো একটা restaurant-এ কিছু waiter আগে থেকেই ready আছে। নতুন customer এলে যে waiter free থাকে সে service দেয়।

---

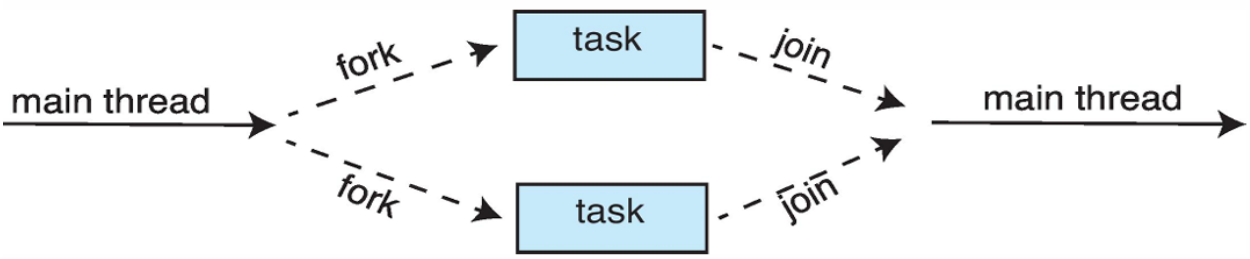
# 2. Fork-Join Model

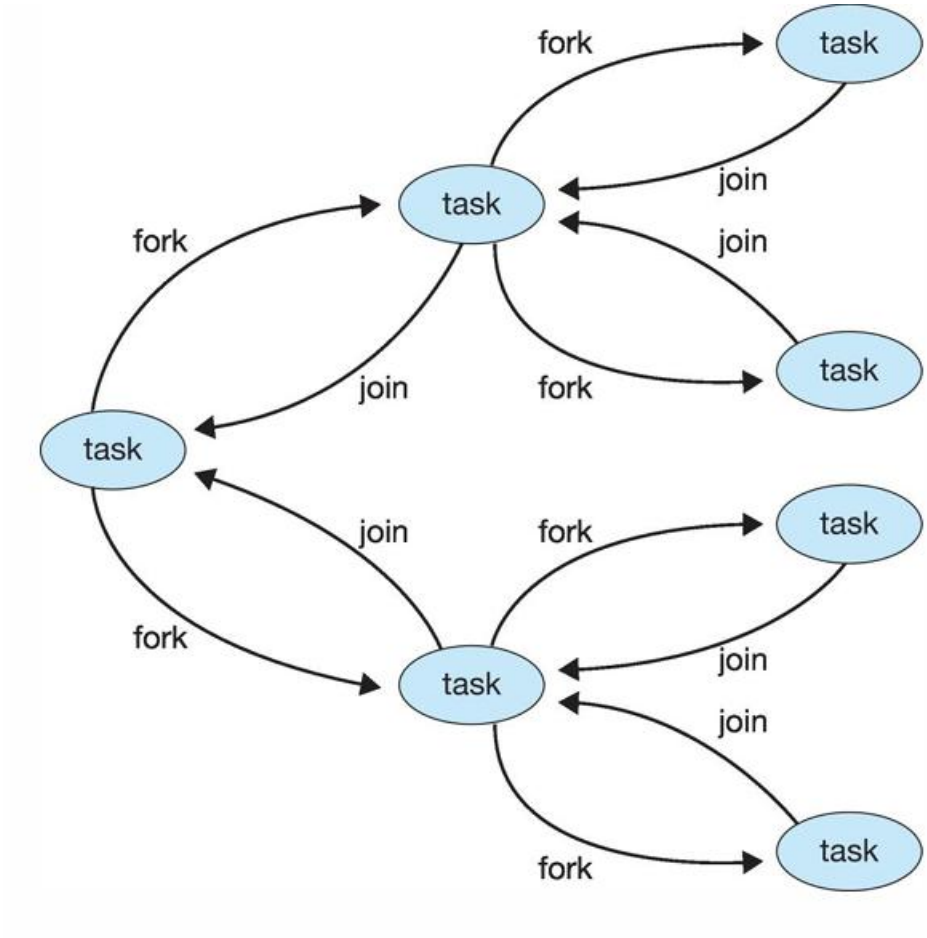
In the **fork-join model**, a parent thread creates multiple child threads (fork), and after completing their tasks, the child threads terminate and combine again (join).

এখানে একটি main (parent) thread কাজকে ভাগ করে ছোট ছোট অংশে child threads-এ দেয়।

### Working Principle

1. Parent thread divides the task.
2. Child threads execute concurrently.
3. Results are merged after completion.




### Advantage

- Efficient parallel processing

---

## উদাহরণ

ভিডিও rendering বা large file processing.

# 3. Grand Central Dispatch (GCD)

**Grand Central Dispatch (GCD)** is a technology developed by Apple for automatic thread management.( GCD হলো Apple-এর একটি system যা automatic thread management করে।)

### Features

- Uses task queues
- Automatically schedules threads
- Balances workload across CPU cores

### Advantages

- Simplifies multithreaded programming
- Improves system performance
- Efficient CPU utilization

### উদাহরণ

iPhone বা macOS app যেখানে background কাজ (download, update) নিজে থেকেই manage হয়

---

# Conclusion

Thread creation requires fewer resources than process creation because threads share the same memory and resources. Implicit threading techniques such as thread pools, fork-join, and Grand Central Dispatch improve performance and simplify concurrent programming by automatically managing threads.

---

---

**5(b)** Define thread. Identify the basic components of a thread, and contrast threads and processes. **(4)(19-20)**

A **thread** is the smallest unit of execution within a process.

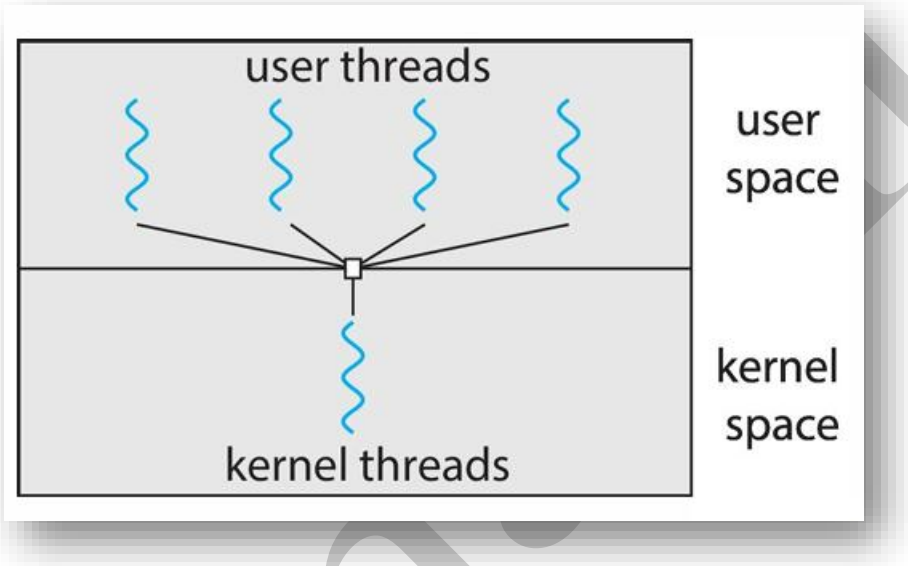
It represents a single sequence of instructions that can be scheduled and executed by the CPU. Multiple threads can exist within the same process, and they share the process's resources such as memory, code, and files, but each thread has its own program counter, register set, and stack.

In simple terms, a thread is a lightweight part of a process that performs a specific task.

**5(c)** Draw multi-threading models. Describe the major benefits and significant challenges of designing multi-threaded processes. **(3)(19-20)**

# 1. Many-to-One Model

- Many user-level threads map to a single kernel thread.
- If one thread blocks, the whole process blocks.




**User-level thread হলো এমন thread যেগুলো পুরোপুরি user space-এ manage করা হয়। Operating system সরাসরি এগুলো সম্পর্কে জানে না।
** Kernel-level thread হলো এমন thread যেগুলো সরাসরি Operating System kernel দ্বারা manage করা হয়।

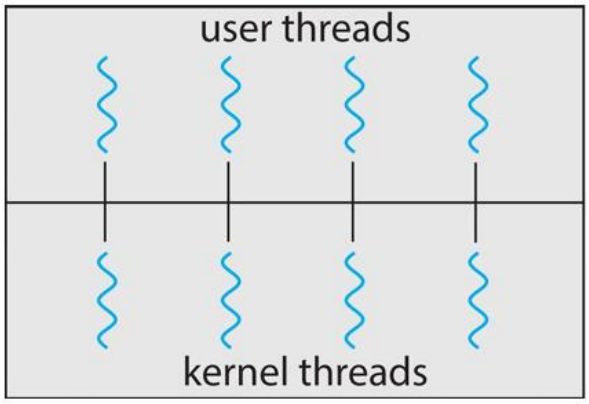
---

# 2. One-to-One Model

- Each user thread maps to a separate kernel thread.
- Better concurrency, but higher overhead.

Examples

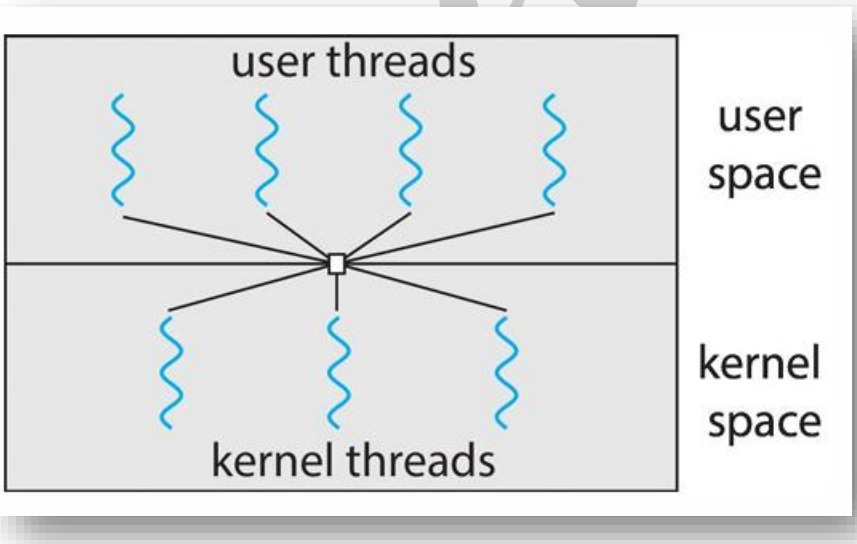
- Windows
- Linux




---

# 3. Many-to-Many Model

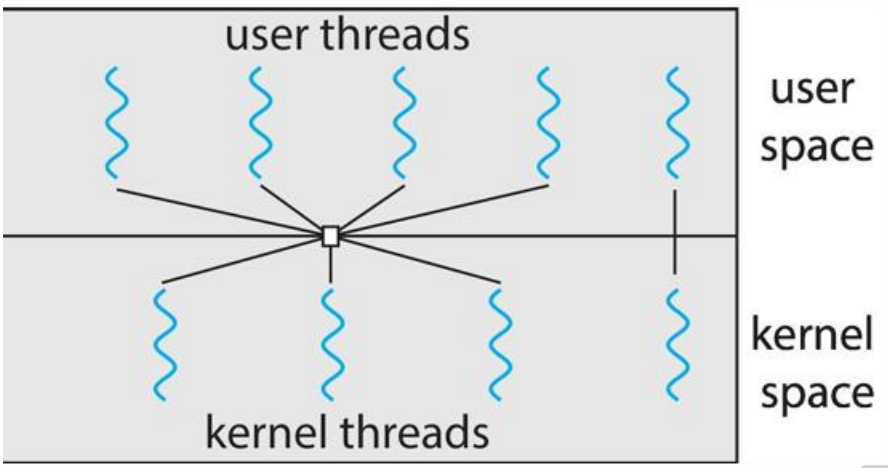
- Many user threads are multiplexed over many kernel threads.
- Balances performance and flexibility.




---

# 4. Two-Level Model (Hybrid)

- Similar to many-to-many, but allows some threads to be bound to specific kernel threads.




---

# Summary

| Model | Key Idea | Limitation |
|---|---|---|
| Many-to-One | Many user threads → 1 kernel thread | No true parallelism |
| One-to-One | 1 user thread → 1 kernel thread | High overhead |
| Many-to-Many | Many user threads → many kernel threads | Complex |
| Two-Level | Hybrid + some bound threads | More complex design |

---

**5(d)** Describe about Java Threads. Illustrate different approaches to implicit threading, including thread pools, fork-join, and Grand Central Dispatch. **(4)**

# Java Threads

A **Java thread** is a lightweight sub-process that allows a program to perform multiple tasks simultaneously. Java supports multithreading to improve CPU utilization and program performance.

A thread in Java is represented by the `Thread` class or the `Runnable` interface.

---

# Features of Java Threads

- Enables concurrent execution
- Improves responsiveness
- Shares memory among threads
- Useful for multitasking applications

---

# Creating Threads in Java

There are two main ways to create threads in Java.

---

# 1. Extending the Thread Class

In this method, a class extends the `Thread` class and overrides the `run()` method.

---

# 2. Implementing the Runnable Interface

In this method, a class implements the `Runnable` interface.

# Advantages of Java Threads

- Faster execution
- Better CPU utilization
- Concurrent task processing
- Improved application responsiveness

# Challenges of Java Threads

- Synchronization problems
- Race conditions
- Deadlocks
- Difficult debugging

---

# Conclusion

Java threads enable concurrent execution of multiple tasks within a program. Threads can be created using the `Thread` class or `Runnable` interface, making Java applications faster and more efficient.

---

---

**What are two differences between user-level threads and kernel-level threads? (Book's Exercise 4.4) ★ [17-18 FINAL]**

# Differences Between User-Level Threads and Kernel-Level Threads

| Feature | User-Level Threads | Kernel-Level Threads |
|---|---|---|
| Management | Managed by user-level thread library | Managed directly by the operating system kernel |
| Switching Speed | Faster context switching | Slower due to kernel involvement |
| System Call Requirement | No kernel mode switch needed | Requires kernel mode operations |
| Blocking Problem | One blocking thread may block all threads | One blocked thread does not block others |
| Parallelism | Cannot fully utilize multiple CPUs | Can run on multiple CPUs simultaneously |

---

**Define data and task parallelism. ★ [18-19 FINAL]**

# Data Parallelism

**Data parallelism** means dividing a large dataset into smaller parts and performing the same operation on each part simultaneously using multiple processors or threads.

Example

- Splitting a large image into sections and processing all sections at the same time.

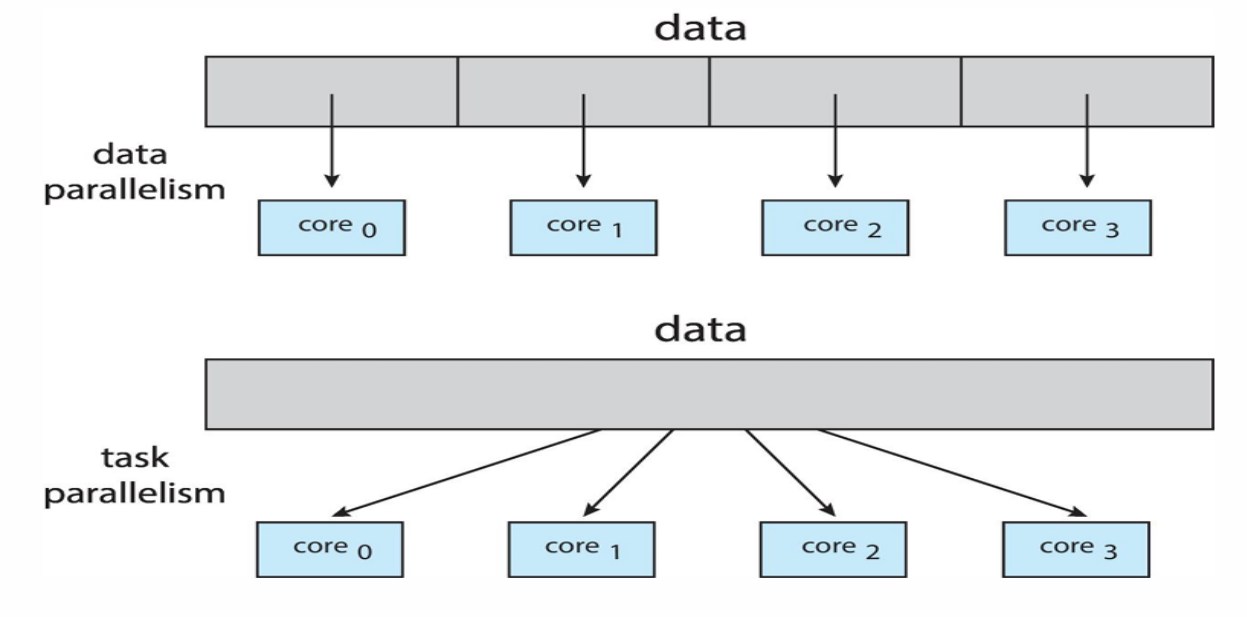
---

# Task Parallelism

**Task parallelism** means dividing a program into different tasks or functions and executing them concurrently.

Example

- One thread handles user input while another thread performs calculations.




# Difference Between Data and Task Parallelism

| Feature | Data Parallelism | Task Parallelism |
|---|---|---|
| Focus | Data division | Task division |
| Operation | Same operation on different data | Different operations simultaneously |
| Example | Array processing | Multiple independent tasks |

**Explain Amdahl's Law. Calculate the speedup gain of an application that has a 60 percent parallel component for two processing cores and four processing cores. (Book's Exercise 4.2) ★★ [16-17 FINAL] [18-19 FINAL]**

# Amdahl's Law

**Amdahl's Law** states that the overall performance improvement gained from parallel processing is limited by the portion of the program that must be executed serially.

It is used to calculate the maximum speedup achievable using multiple processors.

Amdahl's Law বলে যে parallel processing ব্যবহার করলেও program-এর যেই অংশ serially (একটার পর একটা) execute করতে হয়, সেটিই overall performance improvement সীমাবদ্ধ করে।

অর্থাৎ,
program-এর সব অংশ parallel করা যায় না। যেই অংশ parallel করা যায় না, সেটাই speedup-এর limit নির্ধারণ করে।

---

# সহজভাবে বুঝি

ধরো একটি program-এর:

- 80% parallel করা যায়
- 20% অবশ্যই serially run করতে হবে

তুমি যত বেশি processor ব্যবহার করো না কেন, ওই 20% serial অংশের জন্য program পুরোপুরি unlimited speed পাবে না।

## Formula of Amdahl's Law

$$\text{Speedup} = \frac{1}{(1 - P) + \frac{P}{N}}$$

Where:

- $P$ = Parallel portion of the program
- $1 - P$ = Serial portion
- $N$ = Number of processing cores

## Given

- Parallel portion, $P = 60\% = 0.6$
- Serial portion = $1 - 0.6 = 0.4$

## 1. Speedup for 2 Processing Cores

$$\text{Speedup} = \frac{1}{0.4 + \frac{0.6}{2}}$$

$$= \frac{1}{0.4 + 0.3}$$

$$= \frac{1}{0.7}$$

$$= 1.43$$

**Answer**

For **2 cores**, the speedup gain is approximately:

$$\boxed{1.43}$$

## 2. Speedup for 4 Processing Cores

$$\text{Speedup} = \frac{1}{0.4 + \frac{0.6}{4}}$$

$$= \frac{1}{0.4 + 0.15}$$

$$= \frac{1}{0.55}$$

$$= 1.82$$

**Answer**

For **4 cores**, the speedup gain is approximately:

$$\boxed{1.82}$$

**Under what circumstances does a multithreaded solution using multiple kernel threads provide better performance than a single-threaded solution on a single-processor system? (Book's Exercise 4.9) ★ [17-18 FINAL]**

A **multithreaded solution using multiple kernel threads** can provide better performance than a single-threaded solution on a **single-processor system** when the program frequently performs blocking operations such as I/O, waiting for user input, or network communication.

---

# Explanation

In a single-threaded program:

- If the thread performs an I/O operation or waits for an event, the entire process blocks.
- The CPU may remain idle during the waiting period.

In a multithreaded program with multiple kernel threads:

- When one thread blocks, another thread can continue execution.
- The operating system schedules another ready thread on the CPU.
- This improves CPU utilization and responsiveness.

---

# Situations Where Multithreading Performs Better

## 1. I/O-Bound Applications

Examples:

- Web browsers
- File downloading
- Database applications

While one thread waits for disk or network I/O, another thread executes.

---

## 2. Applications Requiring Responsiveness

Examples:

- GUI applications
- Online chat systems

One thread handles the user interface while another performs background tasks.

---

## 3. Concurrent Independent Tasks

If tasks are independent, multiple threads can execute alternately without blocking the whole program.

---

# Example

In a web browser:

- One thread loads a webpage,
- Another thread plays video,
- Another thread responds to user input.

Even on a single CPU, the system appears faster and more responsive because threads run whenever others are waiting.

---

**A system with two dual-core processors has four processors available for scheduling. A CPU-intensive application is running... How many threads will you create to perform the input and output? How many threads will you create for the CPU-intensive portion? Explain. (Book's Exercise 4.16) ★ [17-18 FINAL]**

# Answer

The system has:

- **Two dual-core processors**
- Total available processors:

$$2 \times 2 = 4 \text{ cores}$$

So, the operating system can run **4 threads simultaneously**.

---

# 1. Threads for Input and Output Operations

For input/output (I/O) tasks:

- I/O operations often block while waiting for devices, files, or networks.
- Multiple threads help keep the application responsive.

Recommended

Create:

**1 or 2 I/O threads**

Usually:

- One thread handles input
- One thread handles output

This allows I/O operations to proceed independently of computation.

---

# 2. Threads for CPU-Intensive Portion

CPU-intensive tasks require maximum CPU utilization.

Since there are:
4 processor cores

The ideal number of CPU-intensive threads is:

**4 CPU threads**

This allows:

- One thread per core
- Maximum parallel execution
- Better processor utilization

Creating more CPU-bound threads than cores may increase context-switch overhead without improving performance.

---

# Final Answer

- **I/O threads:** 1–2 threads
- **CPU-intensive threads:** 4 threads

Because the system has 4 cores, four CPU-bound threads can execute simultaneously, while separate I/O threads improve responsiveness and prevent blocking.

---

**Original versions of Apple's mobile iOS operating system provided no means of concurrent processing. Discuss three major complications that concurrent processing adds to an operating system. (Book's Exercise 3.3) ★ [17-18 FINAL]**

# Complications Added by Concurrent Processing in an Operating System

**Concurrent processing** allows multiple processes or threads to execute at the same time. Although it improves performance and responsiveness, it also introduces several complications for the operating system.

---

# 1. Synchronization Problems

When multiple processes or threads share data simultaneously, inconsistent or incorrect results may occur.

## Problem

- Two or more threads may try to access or modify shared data at the same time.

## Example

- Two bank transactions updating the same account balance simultaneously.

## Result

- Data inconsistency or corruption.

## Solution

- Mutexes
- Semaphores
- Monitors

---

# 2. Deadlock

A **deadlock** occurs when two or more processes wait indefinitely for resources held by each other.

## Problem

- Each process waits for another process to release a resource.

## Example

- Process A holds Resource 1 and waits for Resource 2.
- Process B holds Resource 2 and waits for Resource 1.

## Result

- System execution stops for those processes.

## Solution

- Deadlock prevention
- Deadlock avoidance
- Resource allocation control

---

# 3. Race Conditions

A **race condition** happens when multiple threads execute and access shared data unpredictably.

## Problem

- Final output depends on the order of thread execution.

## Example

- Two threads incrementing the same variable simultaneously.

## Result

- Incorrect or unpredictable output.

## Solution

- Proper synchronization techniques

---

# Additional Challenges

- Increased scheduling complexity
- Difficult debugging and testing
- Higher context-switch overhead
- Resource sharing management

---

**Is it possible to have concurrency but not parallelism? Explain. (Book's Exercise 4.13) ★ [17-18 FINAL]**

Yes, it is possible to have **concurrency without parallelism**.

---

# Explanation

## Concurrency

Concurrency means multiple tasks make progress during overlapping time periods. The tasks may not execute exactly at the same instant.

Concurrency মানে একাধিক task একই সময়ে progress করছে।
কিন্তু তারা একই মুহূর্তে execute করতেই হবে এমন না।

## Parallelism

Parallelism means multiple tasks execute literally at the same time using multiple processors or CPU cores.

Parallelism মানে একাধিক task সত্যিকার অর্থে একই সময়ে execute হচ্ছে, সাধারণত multiple CPU/core ব্যবহার করে।

---

# Concurrency Without Parallelism

On a **single-processor system**, only one task can execute at a time. However, the operating system rapidly switches the CPU among multiple tasks using **context switching**.

As a result:

- Multiple tasks appear to run together,
- But only one task is executing at any given instant.

This is concurrency without true parallel execution.

---

# Example

Suppose a single-core CPU runs:

- A music player
- A web browser
- A text editor

The OS switches quickly among these processes:

- Browser executes for a short time
- Then music player executes
- Then text editor executes

This creates the illusion of simultaneous execution.

# Key Point

- **Concurrency** = dealing with multiple tasks at the same time
- **Parallelism** = executing multiple tasks at the exact same time

Therefore:

- Concurrency can exist without parallelism,
- But parallelism always implies concurrency.

# Conclusion

Yes, concurrency without parallelism is possible, especially in single-core systems where the CPU switches rapidly between tasks, allowing multiple tasks to progress without executing simultaneously.